Final Project Report: A Parallel Branch and Bound Solver for the Traveling Salesperson Problem featuring Quality-Equalizing Load Balancing with a Weighted Heuristic

Author/Team: Congqi Huang、Sagar and Lipi

Date: May 11, 2025

Course: CS566

Abstract

This report presents the design, implementation, and comprehensive performance evaluation of a parallel solver for the Traveling Salesperson Problem (TSP), a canonical NP-hard problem in combinatorial optimization. The solver employs the Branch and Bound (B&B) algorithm, parallelized using the Message Passing Interface (MPI). This project systematically evolved from a baseline sequential B&B solver to increasingly sophisticated parallel versions. A key focus was the implementation and refinement of the Quality-Equalizing (QE) dynamic load balancing strategy. This strategy was further enhanced by incorporating a novel weighted heuristic that factors in the progress of partial tours, in addition to their estimated costs, for more nuanced load assessment during balancing operations. This report meticulously documents the architectural details of the various solver iterations, outlines the experimental methodology, presents a detailed analysis of performance metrics (including execution time, speedup, efficiency, and workload distribution statistics), and discusses the empirical findings. The results unequivocally demonstrate the substantial performance benefits of parallelization. More critically, they highlight the efficacy of the QE load balancing scheme, particularly its advanced variant using the weighted progress heuristic, in achieving significant reductions in execution time and improving resource utilization by effectively mitigating load imbalances across processes, especially for larger and more complex TSP instances.

1. Introduction

The Traveling Salesperson Problem (TSP) stands as a cornerstone problem in the field of computer science and operations research. It asks for the shortest possible route that a salesperson must take to visit a given set of cities exactly once, before returning to the original starting city. Despite its simple formulation, the TSP is NP-hard, meaning that the computational effort required to find an optimal solution grows exponentially with the number of cities. This makes exact solutions for large instances intractable with sequential algorithms. Parallel computing paradigms offer a promising avenue to extend the reach of

exact methods by dividing the immense computational workload among multiple processing units.

This project embarked on the development of a robust parallel TSP solver leveraging the Branch and Bound (B&B) algorithmic technique, with parallelism orchestrated through the Message Passing Interface (MPI). The primary objectives of this undertaking were multifaceted:

1. To establish a correct and efficient sequential B&B TSP solver to serve as a performance baseline and for correctness verification.
2. To design and implement a parallel B&B solver using MPI, addressing the challenges of work distribution and global state management.
3. To integrate and thoroughly evaluate the Quality-Equalizing (QE) dynamic load balancing strategy, aiming to enhance parallel efficiency by ensuring an equitable distribution of the search effort.
4. To innovate upon the standard QE approach by devising and implementing a weighted heuristic. This heuristic augments the traditional cost-based quality measure with a notion of "progress," providing a more sophisticated metric for load balancing decisions.
5. To conduct a rigorous performance analysis of all developed solver versions, focusing on execution time, scalability, speedup, parallel efficiency, and the statistical characteristics of workload distribution across MPI processes.

This report will first provide a theoretical grounding on the TSP and the B&B algorithm. Subsequently, it will delve into the specific implementation details of each solver variant, from the sequential baseline to the advanced QE model with the weighted heuristic. The experimental setup and methodology will be clearly defined, followed by an in-depth presentation and critical analysis of the empirical results. The report will conclude with a summary of key findings, a discussion of the encountered challenges, and propositions for potential future enhancements and research directions in this domain.

2. Theoretical Background

2.1 The Traveling Salesperson Problem (TSP)

Formally, given a complete weighted graph $G=(V,E)$, where V is a set of n vertices (cities) and E is a set of edges, with each edge $(u,v) \in E$ having an associated weight $d(u,v)$ (distance), the TSP aims to find a Hamiltonian cycle of minimum total weight. A Hamiltonian cycle is a path that visits each vertex exactly once and returns to the starting vertex. The problem's NP-hard nature implies that no known polynomial-time algorithm can guarantee finding the

optimal solution for all instances.

2.2 The Branch and Bound (B&B) Algorithm

Branch and Bound (B&B) is an exact algorithm paradigm particularly suited for optimization problems. It operates by systematically enumerating candidate solutions in a search tree, pruning large portions of this tree by using upper and lower bounds on the objective function. For the TSP:

- **Branching:** A partial tour (a node in the B&B tree) is extended by choosing an unvisited city as the next city in the tour. This creates new child nodes representing more complete partial tours.
- **Bounding:** For each node (partial tour P), a lower bound (LB(P)) on the cost of any complete tour that can be formed by extending P is computed. A common method involves summing the cost of the partial tour so far with an optimistic estimate of the cost to complete the tour. This estimate is often derived from the Minimum Spanning Tree (MST) cost of the set of unvisited cities, plus the costs of connecting the endpoints of the partial tour to this MST.
- **Pruning:** A Global Upper Bound (GUB) is maintained, representing the cost of the best complete tour found so far. If, for any node P, $LB(P) \geq GUB$, then node P and its entire descendant subtree can be pruned, as no tour derived from P can be better than the current best.
- **Search Strategy:** Nodes are typically stored in a priority queue, ordered by their lower bounds. The node with the smallest lower bound (most promising) is usually selected for expansion next (best-first search).

2.3 Parallel Branch and Bound for TSP

Parallelizing B&B for TSP aims to explore the vast search tree concurrently. This introduces several key considerations:

- **Initial Work Distribution:** Rank 0 typically generates an initial set of subproblems (nodes near the root of the B&B tree) and distributes them among the available MPI processes.
- **Dynamic Load Balancing:** As the search progresses, some processes might exhaust their subproblems (become idle) while others still have a large backlog in their local priority queues. Dynamic load balancing strategies are crucial to redistribute work from busy processes to idle ones to maintain high utilization and reduce overall execution time.
- **Global Upper Bound (GUB) Management:** The GUB must be shared efficiently

among all processes. When a process finds a new, better complete tour, this new GUB needs to be communicated to all other processes as quickly as possible to maximize pruning effectiveness globally. This is often achieved via MPI_Allreduce.

- **Termination Detection:** A distributed algorithm is needed to detect when all processes have no more work to do (their local priority queues are empty) and no messages (e.g., work requests or work transfers) are in transit. This signals that the global optimum (represented by the final GUB) has been found.

2.4 Quality-Equalizing (QE) Load Balancing

QE is a decentralized, dynamic load balancing strategy particularly suited for tree-search algorithms like B&B. In this scheme:

- Each process maintains a local priority queue of B&B nodes.
- Processes periodically exchange "quality" information with their neighbors (often in a ring topology). This quality is typically the lower bound of the S-th best node in their local queue (where S is a small integer, e.g., QE_SPAN_S = 3). This S-th best node's bound acts as a threshold.
- If a process becomes idle or finds that its own best work is significantly worse than a neighbor's advertised threshold, it can send a work request to that neighbor.
- A donor process, upon receiving a request, can send a portion of its work (e.g., nodes ranked 2nd to S-th) to the requester. The goal is to "equalize" the quality of work across processes, preventing some from being starved while others are overwhelmed.

2.5 Weighted Heuristic for QE Load Balancing

To further refine the notion of "work quality" in QE, this project explored a weighted heuristic. Instead of relying solely on the estimated_cost (lower bound) of a B&B node, the weighted heuristic also considers the "progress" of the partial tour represented by the node. The rationale is that a very shallow node (low progress), even if it has a good estimated cost, might lead to a vast subtree, whereas a deeper node (high progress) with a similar estimated cost might be closer to a solution or a point where its subtree can be quickly explored or pruned.

The implemented weighted heuristic is:

$$W(N)=LB(N)+(LB(N)\times WEIGHT_PROGRESS_PENALTY\times progress_ratio(N))$$

where LB(N) is the standard lower bound (estimated cost) of node N, WEIGHT_PROGRESS_PENALTY is a configurable constant, and progress_ratio(N) is calculated as (number_visit_city - 1) / (total_cities - 1).

This formula applies a penalty to nodes that are further along their path (higher progress_ratio). A higher penalty makes these nodes appear "more costly" or "less attractive" from a weighted heuristic perspective. This influences which nodes are shared (donors might prefer to share shallower nodes) and how "busy" a process appears to its neighbors. The local B&B search still prioritizes nodes by their raw estimated_cost.

3. Implementation Details

The project involved the development of several C++ TSP solver versions, each building upon or varying from the previous ones. All implementations utilize common data structures: CityNode (for city coordinates), CityGraph (for problem instance data and distance matrix), and BBNode (for representing states in the B&B search tree).

3.1 Core Data Structures

- CityNode: A simple struct holding int x and int y coordinates.
- CityGraph: Contains int n_city, an array CityNode city[MAXSIZE], and a 2D array float dis[MAXSIZE][MAXSIZE] for pre-calculated Euclidean distances. The MAXSIZE constant defines the compile-time limit on the number of cities.
- BBNode: This crucial struct encapsulates the state of a partial tour. It includes:
    - int number_visit_city: The number of cities currently in the path.
    - float cost_so_far: The actual cost of the partial path traversed.
    - float mst_cost_val: The MST-based lower bound for completing the tour from the current state.
    - int path[MAXSIZE]: An array storing the sequence of city indices visited.
    - int visited[MAXSIZE]: A boolean-like array (0 or 1) indicating which cities have been visited.
    - It provides methods like estimated_cost() (cost_so_far + mst_cost_val), compute_mst_cost(), is_solution(), hamilton_cost(), and visit_city() (to generate child nodes).
- **Local Priority Queue (**local_pq**)**: A std::priority_queue<BBNode*, std::vector<BBNode*>, BBNode::Compare> is used by each process to store pointers to BBNode objects. The BBNode::Compare struct ensures that the queue is a min-heap ordered by BBNode::estimated_cost().

3.2 Sequential Solver (sequential_tsp_bb.cpp)

This version serves as the foundational implementation and performance baseline.

- **Initialization:** Parses the input file to populate city_graph. Creates an initial BBNode (starting at city 0, path: {0}) and computes its MST-based lower bound. This node is pushed onto local_pq.
- **Main Loop:** While local_pq is not empty:
    1. Extract the node with the smallest estimated_cost() from local_pq.
    2. Increment nodes_processed.
    3. **Pruning:** If node->estimated_cost() >= global_upper_bound, discard the node.
    4. **Solution Check:** If node->is_solution(), calculate its hamilton_cost(). If this cost is less than global_upper_bound, update global_upper_bound and increment solutions_found.
    5. **Branching (Expansion):** If not a solution and not pruned, for each unvisited city c, create a child_node by calling node->visit_city(c). If the child_node is valid and child_node->estimated_cost() < global_upper_bound, push it onto local_pq.
    6. Delete the processed node.
- Timing is done using std::chrono::high_resolution_clock.

3.3 Basic Parallel Solver (parallel_tsp_bb.cpp)

This version introduces parallelism using MPI but lacks sophisticated dynamic load balancing.

- **Initialization:**
    - Rank 0 parses the input, creates city_graph, performs an initial limited expansion of the root node to generate a set of distinct subproblems, and updates its nodes_processed count.
    - Rank 0 broadcasts n_city and then the entire city_graph object (using MPI_Bcast(city_graph, sizeof(CityGraph), MPI_BYTE, ...) which, as noted in later versions, can be unsafe due to potential memory layout differences across architectures, though it might work in homogeneous environments). Workers receive this data and reconstruct their city_graph.
    - Rank 0 sends one unique packed BBNode (subproblem) to each other worker process using MPI_Send with TAG_INITIAL_WORK. If Rank 0 runs out of nodes, it sends a 0-byte message.

    - Worker processes receive their initial packed BBNode, unpack it, and push it to their local_pq. If they receive a 0-byte message or if the unpacked node is immediately prunable, they start in an IDLE state.

- BBNode **Serialization:** Uses MPI_Pack and MPI_Unpack to serialize/deserialize BBNode objects into/from a character buffer for transmission. Functions pack_bbnode and unpack_bbnode handle this.

- **Main Loop (**run_parallel_bb_no_qe**)**:

    - Each process runs a B&B loop similar to the sequential version on its local_pq.

    - **GUB Synchronization:** global_upper_bound is updated locally when a better solution is found. Periodically (GUB_UPDATE_INTERVAL), broadcast_global_upper_bound() is called, which uses MPI_Allreduce(MPI_MIN) to find the minimum GUB across all processes and update everyone's local GUB.

    - **State Management:** Processes are either ACTIVE (processing from local_pq) or IDLE (empty local_pq).

    - **Termination Detection (Simplified):**

        - When a process becomes IDLE, it calls check_termination_condition().

        - check_termination_condition(): If IDLE and local_termination_signal_sent is false, it sends a TAG_TERM_CHECK message (containing its rank) to Rank 0 and sets local_termination_signal_sent = true. Rank 0 itself also participates by incrementing terminated_count if it becomes idle.

        - handle_incoming_messages(): Only Rank 0 listens for TAG_TERM_CHECK messages. Upon receiving one, it increments a global terminated_count.

        - Periodically, Rank 0 checks if terminated_count >= psize. If true, it sets a globally_terminated flag to true. This flag is then broadcast to all processes using MPI_Bcast.

        - All processes exit their main loop when globally_terminated becomes true.

- **Final Result Aggregation:** After the B&B loop, worker processes send their final

local global_upper_bound to Rank 0 using TAG_FINAL_GUB. Rank 0 collects these and determines the true global minimum. nodes_processed counts are gathered by Rank 0 using MPI_Gather.

3.4 Parallel Solver with PQ Size Cutting (parallel_tsp_bb_cutting.cpp)

This is a modification of parallel_tsp_bb.cpp.

- Introduces a MAX_PQ_SIZE_LIMIT constant.
- Before pushing any new BBNode (initial node, or child node during expansion) onto local_pq, the code checks if local_pq.size() < MAX_PQ_SIZE_LIMIT.
- If the queue is full, the node is *not* pushed and is instead deleted. A counter nodes_discarded_pq_full tracks these discards.
- The primary intent of this version is to prevent Out-Of-Memory (OOM) errors on very large problems by capping memory usage for the priority queue, fully acknowledging that this heuristic cutting will very likely lead to suboptimal or incorrect final TSP costs because promising unexplored paths might be discarded.

3.5 Parallel Solver with QE Load Balancing (Base: parallel_tsp_qe_serialization.cpp, Evolved to parallel_tsp_qe.cpp with Weighted Heuristic)

This represents the most sophisticated family of solvers developed, focusing on dynamic load balancing. The version parallel_tsp_qe_serialization.cpp uses MPI_Pack/MPI_Unpack for BBNodes similar to parallel_tsp_bb.cpp. The version referred to as parallel_tsp_qe.cpp (and culminating in the logic for parallel_tsp_qe_weighted_full.cpp) evolved to potentially use direct struct send for BBNodes (if architectures are homogeneous and struct is POD-like, which BBNode largely is) but crucially adopted a safer component-wise broadcast for CityGraph.

Key features of the QE implementation (focusing on the advanced weighted heuristic version):

- **Safer** CityGraph **Broadcast:** Rank 0 broadcasts city_graph->n_city, then the city_graph->city array, then the city_graph->dis matrix (the full MAXSIZE*MAXSIZE block) component-wise. This is more robust than broadcasting the sizeof(CityGraph).
- BBNode **Transmission:** Assumed to be direct MPI_Send of sizeof(BBNode) for the parallel_tsp_qe.cpp (weighted) version based on prior interactions where this was generated, though parallel_tsp_qe_serialization.cpp explicitly shows a pack/unpack alternative.

- **State Management (**ProcessState**):** Processes can be ACTIVE, IDLE, or TERM_SENT.
- **Neighbor Communication:** Processes are arranged in a logical ring. neighbor_ranks[0] is previous, neighbor_ranks[1] is next.
- send_threshold_cost()**:**
    - Called periodically or when state changes.
    - Determines a threshold. In the weighted version, this is based on the weighted_heuristic_value() of the S-th best node in its local_pq (or the best node if fewer than S nodes). If the PQ is empty, threshold is INF.
    - Sends this (weighted) threshold to its two neighbors using MPI_Send with TAG_QE_THRESHOLD.
- perform_load_balancing_check()**:**
    - Calls send_threshold_cost().
    - Calculates its own "busyness" or "work quality" using my_best_weighted_val = local_pq.empty() ? INF : local_pq.top()->weighted_heuristic_value().
    - Compares my_best_weighted_val with neighbor_thresholds[] (which now store received weighted thresholds).
    - If local_pq is empty OR my_best_weighted_val is greater (worse) than a neighbor's threshold, and it hasn't recently requested from that neighbor, it calls request_work(neighbor_rank).
- request_work()**:** Sends a TAG_QE_WORK_REQUEST message (containing its own rank) to the target neighbor. Sets requested_work_from[neighbor_index] = true.
- send_work()**:**
    - Called upon receiving a TAG_QE_WORK_REQUEST.
    - If local_pq.size() > WORK_TRANSFER_COUNT (has surplus work):
        - Temporarily removes its best node.
        - Pops WORK_TRANSFER_COUNT nodes (which are the 2nd to (S)-th best) from local_pq.
        - Puts its best node back.
        - Sends these extracted nodes (one by one, after packing if using

serialization, or directly if using struct send) to the requester using TAG_QE_WORK_TRANSFER.

    - Deletes its local copies of the sent nodes.
    - If it was IDLE or TERM_SENT when sending work, it sets received_term_confirm[neighbor_index_of_requester] = false, indicating it needs a confirmation that the work was "in flight" and processed, crucial for the termination protocol.

- handle_incoming_messages()**:** A non-blocking loop (MPI_Iprobe, then MPI_Recv) to process various message types:
    - TAG_QE_THRESHOLD: Updates neighbor_thresholds[] and resets requested_work_from[] for that neighbor.
    - TAG_QE_WORK_REQUEST: Calls send_work().
    - TAG_QE_WORK_TRANSFER: Receives (and unpacks if serialized) BBNode. If the node's estimated_cost() is better than global_upper_bound, pushes it to local_pq. If the receiver was IDLE or TERM_SENT, it calls send_wakeup_signal() and transitions to ACTIVE.
    - TAG_TERM_CHECK: If Rank 0, increments terminated_count.
    - TAG_TERM_WAKEUP: If Rank 0, decrements terminated_count (if >0). If any rank had local_termination_signal_sent = true, it resets it to false and, if TERM_SENT, transitions to IDLE.
    - TAG_TERM_CONFIRM: Sets the corresponding received_term_confirm[] flag to true.

- **Termination Protocol (**check_termination_condition**,** send_termination_signal**,** send_wakeup_signal**):**
    - check_termination_condition(): Called when IDLE. A process decides to terminate if:

1. It is IDLE and local_pq is empty.
2. It has not already sent a termination signal.
3. Its neighbors do not appear to have useful work (i.e., neighbor_thresholds[k] >= global_upper_bound for both neighbors. *Note: This compares a weighted threshold to an unweighted GUB, a point for discussion*).
4. It has received confirmations for any work it might have sent while inactive

(received_term_confirm[0] and received_term_confirm[1] are true).

  - send_termination_signal(): If conditions are met, sets current_state = TERM_SENT, local_termination_signal_sent = true, and broadcasts TAG_TERM_CHECK to all *other* processes. If Rank 0 sends this, it also increments its own terminated_count.
  - send_wakeup_signal(): Called when a process transitions from IDLE/TERM_SENT to ACTIVE (e.g., after receiving work). It resets local_termination_signal_sent, sets current_state = ACTIVE, and broadcasts TAG_TERM_WAKEUP to all other processes. It also sends any pending TAG_TERM_CONFIRM messages if received_term_confirm[] flags were false (meaning it had sent work while inactive and is now confirming its own active status related to that transfer).
  - **Global Termination:** Rank 0 checks if terminated_count >= psize AND its own local_pq is empty. If so, it sets a globally_terminated flag and broadcasts this to all processes, causing them to exit their main B&B loop.

- BBNode::weighted_heuristic_value() **and its integration (as described in Section 2.5):** This function is called within send_threshold_cost to determine the value of the threshold being sent, and within perform_load_balancing_check for a process to evaluate its own best work's quality using this weighted measure. The local priority queue local_pq continues to be sorted based on the raw estimated_cost.

4. Experimental Setup and Methodology

- **Hardware and Software Environment:**
  - The experiments were conducted on a high-performance computing cluster.
  - The SLURM Workload Manager (slurm/lakeshore/23.02.4) was used for job submission and resource allocation.
  - Software stack: GCC compiler version 12.3.0 (GCC/12.3.0) and OpenMPI version 4.1.5 compiled with GCC 12.3.0 (OpenMPI/4.1.5-GCC-12.3.0).
  - The C++ code was compiled with -O2 optimization and the -std=c++11 standard, linked with the math library (-lm).
- **MPI Configuration:**
  - Parallel jobs were launched using srun --mpi=pmix -n $NP, where $NP is the total number of MPI processes.

    - For the results presented, a configuration of 4 nodes with 8 MPI tasks per node was typically used, resulting in a total of NP=4×8=32 processes.
    - Resource allocation per job: 2 hours 30 minutes maximum wall time, 52GB memory.

- **Problem Instances:**
    - The TSP solver was tested on a series of problem instances with varying numbers of cities (N): 8, 15, 28, 34, 36, and 40.
    - Input city data was read from a file named "Input.txt" (as defined by the FILENAME macro in the source code). The specific datasets used for these N values are assumed to be consistent across runs.

- **Solver Versions Tested (as per CSV results):**

1. Sequential: Baseline sequential Branch and Bound.

2. Parallel: Basic parallel B&B with MPI, no dynamic load balancing, uses MPI_Pack/MPI_Unpack for BBNodes.

3. Parallel-Cut: Similar to Parallel, but with a hard limit on local priority queue size to prevent OOM.

4. QE: Parallel B&B with Quality-Equalizing load balancing. Based on the CSV execution times, this likely refers to a version that might not use explicit MPI_Pack/MPI_Unpack for BBNodes or has other optimizations compared to QE-Serialization.

5. QE-Serialization: Parallel B&B with QE, explicitly using MPI_Pack/MPI_Unpack for BBNode serialization.

6. QE-Weighted-Full: The advanced QE version incorporating the weighted_heuristic_value() for load assessment, with WEIGHT_PROGRESS_PENALTY likely set to 0.15f (a common default used during development).

- **Performance Metrics:**
    - **Execution Time (seconds):** Total wall-clock time for the solver to complete.
    - **Total Nodes Processed (Global):** Sum of nodes_processed by all MPI ranks.
    - **Average Nodes Processed (per Rank):** Total Nodes Processed / psize.
    - **Max Nodes Processed (per Rank):** The highest nodes_processed count among all ranks.
    - **Min Nodes Processed (per Rank):** The lowest nodes_processed count

among all ranks.

- **Max-Min Node Difference:** Max Nodes Processed - Min Nodes Processed, a measure of absolute workload imbalance.
- **Standard Deviation of Nodes Processed:** A statistical measure of workload imbalance across ranks. Lower values indicate better balance.
- (Implicitly) **Speedup and Efficiency:** Can be derived by comparing parallel execution times to the sequential execution time. Speedup Sp=Ts/Tp, Efficiency Ep=Sp/p, where Ts is sequential time, Tp is parallel time with p processes.

5. Results and Detailed Analysis

The performance results, extracted from the provided CSV file (cs566.xlsx - Sheet2.csv), are presented and analyzed below. All parallel experiments were conducted using 32 MPI processes.

5.1 Execution Time Analysis

N (Cities)	Sequential (s)	Parallel (s)	Parallel-Cut (s)	QE (s)	QE-Serialization (s)	QE-Weighted-Full (s)
8	0.001	0.199	0.193	0.236	0.372	-
15	0.006	0.180	0.137	0.857	2.570	-
28	11.011	3.814	3.716	3.500	6.619	3.625
34	113.418	29.705	29.163	25.409	34.408	**15.743**
36	565.100	204.844	202.846	80.427	340.612	**41.278**
40	10525.040	3382.890	2216.610	**322.870**	527.667	1907.836

- **Sequential vs. Parallel:** As expected, for N ≥ 28, all parallel versions significantly outperform the sequential solver. For N=40, the sequential time is over 10,000 seconds, while the best parallel times are orders of magnitude lower. For very small N (8, 15), MPI overhead makes parallel versions slower than the trivial sequential execution.
- **Basic Parallel Performance:** The Parallel version (no QE) shows considerable speedup but also suffers from high workload imbalance (see Section 5.2). The Parallel-Cut version offers marginal or sometimes slightly better performance than Parallel, likely due to reduced memory pressure for larger instances (e.g., N=40, Parallel-Cut is faster), though this comes at the risk of cutting off optimal solution paths.
- **QE vs. Basic Parallel:** The QE strategy consistently outperforms the basic Parallel approach for N ≥ 28.
    - N=28: QE (3.500s) vs Parallel (3.814s) - a modest improvement.
    - N=34: QE (25.409s) vs Parallel (29.705s) - a noticeable improvement.
    - N=36: QE (80.427s) vs Parallel (204.844s) - more than 2.5x speedup due to QE.
    - N=40: QE (322.870s) vs Parallel (3382.890s) - a dramatic >10x speedup, underscoring QE's critical role for large problems.
- **QE-Serialization Overhead:** The QE-Serialization variant, which explicitly uses MPI_Pack/MPI_Unpack for BBNodes, is consistently slower than the QE version (which likely uses direct MPI_Send of the BBNode struct). For N=36, QE-Serialization took 340s vs QE's 80s. This suggests that for the (assumed) homogeneous cluster environment, the overhead of manual packing/unpacking outweighed the benefits or risks of direct struct transmission for BBNode. (Note: The safer component-wise broadcast for CityGraph is a separate, beneficial improvement applied to the advanced QE versions).
- **QE-Weighted-Full Performance:**
    - For N=34, QE-Weighted-Full (15.743s) is the fastest among all versions, significantly improving upon basic QE (25.409s).
    - For N=36, QE-Weighted-Full (41.278s) is also the fastest, nearly halving the time of basic QE (80.427s).
    - For N=28, QE-Weighted-Full (3.625s) is slightly slower than basic QE

(3.500s) but comparable.

- **Anomaly at N=40:** For N=40, QE-Weighted-Full (1907.836s) is substantially *slower* than basic QE (322.870s) and even slower than Parallel-Cut (2216.610s), though still faster than the basic Parallel version. This is a significant deviation from the trend observed at N=34 and N=36.

5.2 Workload Distribution Analysis (Standard Deviation of Nodes Processed)

N (Cities)	Parallel	Parallel-Cut	QE	QE-Serialization	QE-Weighted-Full
8	25	27	27	27	-
15	361	364	361	361	-
28	6029.90	6031.86	1124.41	1325.15	1964.89
34	38472.61	38474.17	27212.65	7366.40	26179.56
36	459252.73	459254.88	162893.77	108116.43	**98033.00**
40	5598368.80	1160438.27	**17278.52**	50557.98	2220761.20

- **QE's Impact on Balance:** For N ≥ 28, all QE variants (QE, QE-Serialization, QE-Weighted-Full) show a dramatic reduction in the standard deviation of nodes processed per rank compared to Parallel and Parallel-Cut. This clearly demonstrates that QE successfully distributes the workload much more evenly.
  - For N=40, QE has a std. dev. of ~17k, while Parallel has ~5.6M, a difference of several orders of magnitude.
- **Weighted Heuristic and Balance:**
  - For N=36, QE-Weighted-Full achieves the lowest standard deviation (98,033) among all versions, indicating superior load balance for this instance, which correlates with its excellent execution time.

- For N=34, its std. dev. (26,179) is comparable to basic QE (27,212) and better than Parallel. The QE-Serialization shows an even better std. dev. here (7,366), though its runtime was worse.
- For N=28, QE-Weighted-Full (1,964) has a higher std. dev. than basic QE (1,124).
- For N=40, QE-Weighted-Full (2.2M) has a significantly higher std. dev. than basic QE (~17k). This correlates with its poorer runtime for N=40 and its higher total node count. The weighted heuristic's penalty on more progressed nodes might cause different work-sharing dynamics. While it can lead to faster overall solutions (as seen for N=34, N=36), it doesn't always guarantee the most statistically even distribution of *node counts*. It might be guiding the search more effectively or differently, even if the raw node counts are less uniform in some cases.

5.3 Total Nodes Processed

N (Cities)	Sequential	Parallel	Parallel-Cut	QE	QE-Serialization	QE-Weighted-Full
8	263	1277	1292	1297	1305	-
15	845	21518	21553	21341	21377	-
28	560809	2612419	2612448	714095	515021	846689
34	3317848	17056367	17056387	4238278	1995171	**2128099**
36	17962824	24726232	24726257	11432140	8559076	**5129744**
40	313123461	358750314	195915207	38695284	37050804	114662706

- Parallel versions generally process more total nodes than the sequential version due to redundant work, speculative exploration before GUBs are fully propagated, or less

efficient global pruning initially.

- QE versions often reduce the total nodes processed compared to basic Parallel for larger N (e.g., N=28, N=34, N=36, N=40 for QE vs Parallel). This indicates that better load balancing leads to more effective global pruning as processes are less likely to deeply explore suboptimal branches when good GUBs are shared and work is distributed from more promising parts of the search tree.
- QE-Weighted-Full processed the fewest total nodes for N=34 and N=36, correlating strongly with its best execution times for these instances. This suggests the weighted heuristic, in these cases, not only balanced load but also guided the collective search more efficiently towards the solution, leading to less overall work.
- For N=40, QE-Weighted-Full processed significantly more nodes (114M) than basic QE (38M) and QE-Serialization (37M). This, combined with its longer runtime, suggests that for this very large instance, the weighted heuristic might have led to a less efficient exploration pattern or increased the processing of nodes that were ultimately not on the optimal path, possibly due to the interaction of the progress penalty with the sheer scale of the search space.

5.4 Discussion of the N=40 Anomaly for QE-Weighted-Full

The performance of QE-Weighted-Full at N=40 (1907s runtime, 114M nodes, std. dev. 2.2M) is notably worse than basic QE (322s, 38M nodes, std. dev. 17k). Potential reasons for this discrepancy include:

1. **Suboptimal** WEIGHT_PROGRESS_PENALTY**:** The fixed penalty factor (e.g., 0.15f) might be poorly suited for N=40. A penalty that is too high could make moderately progressed, but still very good, nodes appear excessively "costly" for sharing, hindering effective load distribution or causing processes to hold onto work that should be shared, or leading to excessive exploration of shallower parts of the tree if deeper promising nodes are overly penalized.
2. **Interaction with Termination Logic:** The check_termination_condition compares the *weighted* neighbor thresholds with the *unweighted* global_upper_bound. If the penalty significantly inflates the threshold, processes might prematurely believe their neighbors have no useful work, potentially impacting the overall search or termination dynamics.
3. **Search Space Characteristics:** The specific structure of the search space for N=40 might interact unfavorably with this particular heuristic. The penalty might discourage exploration of critical, deep paths too early.

4. **Single Data Point:** This could be an outlier run. More runs would be needed to confirm this behavior. The fact that QE-Weighted-Full processed nearly 3 times as many nodes as basic QE for N=40 strongly suggests it was not pruning as effectively or was exploring less fruitful regions of the search space for that specific large problem.

6. Challenges, Limitations, and Future Work

6.1 Challenges Encountered

- **Robust Termination Detection:** Designing and implementing a correct and efficient distributed termination detection algorithm for a dynamic system with asynchronous work transfers and potential race conditions was a significant challenge. The custom protocol used, while functional, involved careful state management (ACTIVE, IDLE, TERM_SENT) and explicit confirmations (TAG_TERM_CONFIRM).
- **Parameter Tuning:** Optimizing parameters such as QE_SPAN_S, WORK_TRANSFER_COUNT, WEIGHT_PROGRESS_PENALTY, and various communication intervals (LB_CHECK_INTERVAL, GUB_UPDATE_INTERVAL) is complex and problem-dependent. The chosen values represent a general configuration.
- **MPI Data Transmission:** Deciding between direct struct transmission (simpler, potentially faster in homogeneous environments, but less safe) and manual serialization with MPI_Pack/MPI_Unpack (safer, more portable, but with overhead) for BBNodes was a design consideration. The final QE-Weighted-Full version conceptually built upon a safer CityGraph broadcast, while BBNode transmission details varied across earlier QE experiments. The results for QE-Serialization suggest pack/unpack overhead for BBNode was not beneficial in the test environment.
- **Debugging Parallel Programs:** Identifying and resolving issues like deadlocks, race conditions, or performance bottlenecks in MPI programs is inherently more challenging than in sequential code. The use of DEBUG_PRINT macros (as seen in parallel_tsp_qe_serialization.cpp and other files) is essential.

6.2 Limitations of the Current Work

- **Fixed** WEIGHT_PROGRESS_PENALTY**:** The penalty factor in the weighted heuristic is static. An adaptive mechanism might yield better results across diverse problem instances.
- **Termination Condition Sensitivity:** The comparison of weighted neighbor thresholds to an unweighted GUB in the termination logic is a potential area for

refinement.

- **Homogeneity Assumption for Direct Struct Send:** If direct MPI_Send of BBNode structs was used in the fastest QE-Weighted-Full variant, its portability to heterogeneous systems might be limited (though CityGraph broadcast was made safer).
- **Scalability Testing:** While tested on 32 processes, a more extensive scalability study across a wider range of process counts would be beneficial.
- **Input File Format:** The parser is tailored for a specific TSPLIB-like format (EUC_2D, NODE_COORD_SECTION).

6.3 Future Work and Potential Enhancements

- **Adaptive Weighted Heuristic:** Develop a mechanism where WEIGHT_PROGRESS_PENALTY can be adjusted dynamically based on search progress or problem characteristics. Alternatively, explore different non-linear weighting functions for progress.
- **Refined Termination Logic:** Re-evaluate the comparison between weighted thresholds and the unweighted GUB in check_termination_condition. One approach could be for processes to also share their S-th *unweighted* cost, or for a receiver of a weighted threshold to try and "de-weight" it (if possible and meaningful) for comparison against GUB.
- **Hybrid MPI+OpenMP Parallelism:** For multi-core nodes, employ OpenMP for intra-node parallelism to further subdivide the work handled by each MPI process, potentially improving CPU utilization and reducing the number of MPI messages.
- **Advanced Lower Bounding Techniques:** Investigate and implement more powerful lower bounding techniques than MST, such as the Held-Karp lower bound (1-tree relaxation), which could lead to more aggressive pruning and faster solutions, albeit at a higher cost per node.
- **Work Stealing Variants:** Compare the current QE (work pulling/requesting) approach with work-stealing strategies where idle processors actively take work from busy ones.
- **Solution Quality Verification:** Systematically run all solver versions on benchmark TSP instances with known optimal solutions to verify correctness and, for versions like Parallel-Cut, to quantify any deviation from optimality.
- **Comprehensive Scalability Analysis:** Test the best-performing versions (e.g., QE-Weighted-Full) on a significantly larger number of cores (e.g., 64, 128, 256) to assess

strong and weak scaling properties.

- **GUI for Visualization:** Develop a simple graphical interface to visualize the B&B tree search or the progression of the TSP tour finding.

7. Conclusion

This project successfully designed, implemented, and evaluated a series of parallel Traveling Salesperson Problem solvers using the Branch and Bound algorithm and MPI. The journey from a sequential solver to a sophisticated parallel version with Quality-Equalizing (QE) load balancing and a novel weighted heuristic demonstrated significant improvements in performance and resource utilization.

The key findings are:

1. **Parallelism is Essential:** MPI-based parallelization provides orders-of-magnitude speedup over sequential execution for non-trivial TSP instances.
2. **Dynamic Load Balancing is Crucial:** The Quality-Equalizing (QE) strategy dramatically improves workload distribution (as evidenced by lower standard deviations in nodes processed per rank) and reduces overall execution time compared to basic parallel implementations without dynamic load balancing.
3. **Weighted Heuristic Shows Promise:** The introduction of a weighted_heuristic_value that incorporates path progress into the QE load assessment mechanism led to the best execution times for several medium-to-large problem instances (N=34, N=36), suggesting it can guide the distributed search more effectively. It also achieved the best load balance for N=36.
4. **Serialization Overhead Matters:** For BBNode transmission, direct struct send (in a compatible environment) appeared faster than manual MPI_Pack/MPI_Unpack in the tested setup, although component-wise broadcast for complex structs like CityGraph remains a good practice for robustness.
5. **Parameter Sensitivity:** The performance of advanced heuristics (like the weighted QE) can be sensitive to parameter choices (e.g., WEIGHT_PROGRESS_PENALTY) and may require tuning for different problem scales or characteristics, as suggested by the N=40 anomaly.

The project underscores the power of combining sound algorithmic principles like Branch and Bound with effective parallelization techniques and intelligent dynamic load balancing to tackle computationally formidable problems. The developed QE-Weighted-Full solver, despite needing further investigation for very large instances, represents a promising

advancement in the QE framework by incorporating a more nuanced view of work quality. Future work can build upon this foundation to further enhance its robustness, scalability, and overall efficiency.

8. References

- Course materials for CS566 covering Parallel Computing, MPI, Branch and Bound, and TSP.
- General literature on Quality-Equalizing load balancing strategies in parallel tree search.
- The C++ source code files provided: sequential_tsp_bb.cpp, parallel_tsp_bb.cpp, parallel_tsp_bb_cutting.cpp, parallel_tsp_qe.cpp, parallel_tsp_qe_serialization.cpp.
- The conceptual logic for parallel_tsp_qe_weighted_full.cpp as developed throughout the interaction.
- The experimental results CSV file: cs566.xlsx - Sheet2.csv.
- The SLURM job submission script: submit-job-qe_weighted_full.sh.

Appendix A: SLURM Job Submission Script (submit-job-qe_weighted_full.sh)

Bash

```
#!/bin/bash


#SBATCH --partition=batch                # Partition name (check your cluster's config)
#SBATCH --job-name=tsp_qe_job        # Job name
#SBATCH --nodes=4                            # Number of nodes requested
#SBATCH --ntasks-per-node=8            # Number of MPI processes per node
#SBATCH --cpus-per-task=1               # Usually 1 CPU per MPI task
#SBATCH --time=2:30:00                   # Max wall time (HH:MM:SS)
#SBATCH --mem=52G                                # Memory requested (e.g., 52 Gigabytes)
#SBATCH --output=tsp_qe_job_%j.log # Standard output log file (%j expands to job ID)
```

```
#SBATCH --error=tsp_qe_job_%j.err    # Standard error log file
#SBATCH --mail-user=hcong@uic.edu # Optional: Your email for notifications
#SBATCH --mail-type=END,FAIL             # Optional: Email on job end or fail

# --- Environment Setup ---
echo "Loading modules..."
module purge # Start with a clean environment
module load slurm/lakeshore/23.02.4 # Slurm module
module load GCC/12.3.0                        # GCC compiler
module load OpenMPI/4.1.5-GCC-12.3.0 # OpenMPI library
echo "Modules loaded."
echo "Currently loaded modules:"
module list # Print loaded modules to log for verification

# --- Compilation ---
# Compile the C++ code using the MPI compiler wrapper (mpic++)
# -lm links the math library
# -O2 enables optimizations
# -std=c++11 specifies the C++ standard
echo "Compiling parallel_tsp_qe_weighted_full.cpp..."
mpic++ parallel_tsp_qe_weighted_full.cpp -o parallel_tsp_qe_weighted_full -lm -O2 -
std=c++11

# Check if compilation was successful
if [ $? -ne 0 ]; then
    echo "Compilation failed!"
    exit 1
```

```
fi
echo "Compilation successful."

# --- Execution ---
# Calculate total number of processes based on Slurm allocation
if [ -n "$SLURM_NNODES" ] && [ -n "$SLURM_NTASKS_PER_NODE" ]; then
    NNODES=$SLURM_NNODES
    NTASKS_PER_NODE=$SLURM_NTASKS_PER_NODE
    NP=$((NNODES * NTASKS_PER_NODE)) # Total number of MPI processes
else
    echo "Error: Slurm environment variables (SLURM_NNODES, SLURM_NTASKS_PER_NODE) not found."
    echo "Cannot determine NP (total number of processes)."
    exit 1
fi

echo "Running job on $NNODES nodes with $NTASKS_PER_NODE tasks per node (Total $NP processes)"

# Run the MPI executable using srun (recommended for Slurm environments)
# --mpi=pmix specifies the MPI type for srun to coordinate with Slurm
echo "Launching with srun..."
srun --mpi=pmix -n $NP ./parallel_tsp_qe_weighted_full

# The mpirun command is commented out as srun is generally preferred with Slurm
# echo "Launching with mpirun..."
# mpirun -np $NP ./parallel_tsp_qe_weighted_full
```

```
echo "Job finished."
```